

Matthew Tran

Breaking Ciphers and Hashes: Cipher and MD5 Collision Lab

The purpose of this lab was to learn how basic encryption and hashing systems work and why some of them are no longer secure. In the first part of the lab, I worked with a substitution cipher and used frequency analysis to see how encrypted text can be cracked. In the second part, I performed an MD5 collision attack to understand why MD5 is considered unsafe today. This lab showed how small weaknesses in cryptography can lead to serious security problems.

Tools Used

- Python
- GCC
- md5collgen
- Wireshark Virtual Machine
- Linux terminal commands (tr, cat, head, tail, md5sum)

Skills Learned

- Creating and breaking substitution ciphers
- Using frequency analysis to crack encrypted text
- Understanding MD5 hash collisions
- Working with binary files
- Recognizing why outdated cryptographic algorithms are insecure

Task 1) Creating a Random Substitution Key

1. Opened the terminal inside my manually created virtual machine.
2. Used vim to create a Python file named key.py.
3. Wrote Python code that used the random.sample() function to shuffle the lowercase alphabet.
4. Printed both the original alphabet and the generated substitution key to the terminal.
5. Ran the script using python3 key.py to generate the key.

```
matt@matt-VirtualBox:~$ vim key.py
```

```
matt@matt-VirtualBox: ~  
  
import random  
  
s = "abcdefghijklmnopqrstuvwxyz"  
k = ''.join(random.sample(s, len(s)))  
print(f'Alphabets, s = "{s}"')  
print(f'Substitution key, k = "{k}"')
```

```
matt@matt-VirtualBox:~$ python3 key.py  
Alphabets, s = "abcdefghijklmnopqrstuvwxyz"  
Substitution key, k = "nleubdwzaifvmxpytcsrhhokgqj"
```

This step created a random substitution key that replaces each letter with another letter. This key is required to encrypt plaintext using a monoalphabetic cipher.

Task 2) Cleaning the Original Text

1. Used the tr command to convert all uppercase letters in article.txt to lowercase.
2. Saved the output to a file named lowercase.txt.
3. Used another tr command to remove punctuation and numbers from the text.
4. Saved the cleaned output to plaintext.txt.

```
matt@matt-VirtualBox:~/Downloads$ tr '[:upper:]' '[:lower:]' < article.txt > lowercase.txt  
matt@matt-VirtualBox:~/Downloads$ tr -cd '[:lower:][:space:]' < lowercase.txt > plaintext.txt
```

```
matt@matt-VirtualBox:~/Downloads$ head -n 20 plaintext.txt
the enigma machine was invented by the german engineer arthur scherbius at the
end of world war i this was not known until when a paper by karl de
leeuw was found that described in detail arthur scherbius changes the
german firm scherbius ritter cofounded by arthur scherbius patented ideas
for a cipher machine in and began marketing the finished product under
the brand name enigma in initially targeted at commercial markets
early models were used commercially from the early s and adopted by
military and government services of several countries most notably nazi
germany before and during world war ii

several different enigma models were produced but the german military models
having a plugboard were the most complex japanese and italian models were
also in use with its adoption in slightly modified form by the german navy
in and the german army and air force soon after the name enigma became
widely known in military circles prewar german military planning emphasized
fast mobile forces and tactics later known as blitzkrieg which depend on
radio communication for command and coordination since adversaries would
likely intercept radio signals messages would have to be protected with
secure encoding compact and easily portable the enigma machine filled that
need
```

The first command I used converted all uppercase letters to lowercase. The second command would remove all characters that weren't a space or lowercase letter. Although everything does look more clumped up, the text also looks much more consistent for any needed analysis.

Task 3: Encrypting the Plaintext

1. Identified the original alphabet and the generated substitution key.
2. Used the `tr` command to map each letter to its encrypted version.
3. Left spaces unchanged during encryption.
4. Saved the encrypted output to `ciphertext.txt`.

```
matt@matt-VirtualBox:~/Downloads$ tr 'abcdefghijklmnopqrstuvwxyz' 'nleudbwzaifv
mxpytcsrghokgjj' < plaintext.txt > ciphertext.txt
```

```

matt@matt-VirtualBox:~/Downloads$ head -n 20 cipher.txt
head: cannot open 'cipher.txt' for reading: No such file or directory
matt@matt-VirtualBox:~/Downloads$ head -n 20 ciphertext.txt
rzd dxawmn mnezaxd kns axodxrdu lq rzd wdcmnx dxwaxddc ncrzhc sezdclahs nr r
dxu pb kpcvu knc a rzas kns xpr fxpkx hxrav kzdx n ynydc lq fncv ud
vddhk kns bphxu rznr udsecaldu ax udrnav ncrzhc sezdclahs eznxwds rzd
wdcmnx bacm sezdclahs carrdc epbphxudu lq ncrzhc sezdclahs ynrdrdu audns
bpc n eayzdc mnezaxd ax nxu ldwnx mncfdraxw rzd baxaszdu ycpuher hxudc
rzd lcnxu xnmd dxawmn ax axaranvvq rncwdrdu nr epmmnceanv mncfdrs
dncvq mpudvs kcdc hsdv epmmnceanvvq bcpm rzd dncvq s nxu nupyrdu lq
mavarncq nxu wpodcxmdxr sdcoads pb sdodcnv ephxrcads mpsr xprnlvq xnja
wdcmnxq ldbpcd nxu uhcaxw kpcvu knc aa

sdodcnv uabbdcdxr dxawmn mpudvs kcdc ycpuherdu lhr rzd wdcmnx mavarncq mpudvs
znoaxwnn yvhwlpncu kcdc rzd mpsr epmyvdg inynxdsd nxu arnvanx mpudvs kcdc
nvsp ax hsd karz ars nupyrax ax svawzrvq mpuabadu bcpm lq rzd wdcmnx xnoq
ax nxu rzd wdcmnx ncmq nxu nac bpced sppx nbrdc rzd xnmd dxawmn ldenmd
kaudvq fxpkx ax mavarncq eacevds ycdknc wdcmnx mavarncq yvnxxaxw dmyznsajdu
bnst mptavd bpceds nxu rneraes vnrde fxpkx ns lvarjfcadw kzaez udydxu px
cnuap epmmhxaenrapx bpc epmmnxu nxu eppcuaxnrapx saxed nuodcsncads kphvu
vafdvq axrdcedyr cnuap sawxnvs mdssnws kphvu znod rp ld ycpdrderdu karz
sdehcd dxepuaxw epmyner nxu dnsavq ypcrnlvd rzd dxawmn mnezaxd bavvdu rznr

```

The command used would change the original used alphabet into the key (abcdefghijklmnopqrstuvwxyz to nleubdwzaifvmxpytcsrhhokgqj). Although all of the original letters and words have been scrambled the spaces still remain the same.

Task 4) Frequency Analysis of the Ciphertext

1. Created a Python file named freq.py using vim.
2. Wrote code to read ciphertext.txt and count letter patterns.
3. Calculated single-letter, bigram, and trigram frequencies.
4. Ran the script to display the most common patterns.

```

matt@matt-VirtualBox:~/Downloads$ vim freq.py

from collections import Counter

with open("ciphertext.txt") as f:
    text = "".join(c for c in f.read().lower() if c.isalpha())

def ngrams(s, n):
    return [s[i:i+n] for i in range(len(s)-n+1)]

for n in (1, 2, 3):
    counts = Counter(ngrams(text, n))
    print("-"*50)
    print(f"{n}-gram (top 20):")
    for gram, freq in counts.most_common(20):
        print(f"{gram}: {freq}")

```

```
matt@matt-VirtualBox:~/Downloads$ python3 freq.py
```

```
-----  
1-gram (top 20):
```

```
d: 188  
n: 142  
x: 120  
a: 118  
c: 109  
r: 107  
p: 89  
s: 86  
u: 70  
v: 63  
m: 62  
e: 59  
z: 56  
w: 36  
h: 31  
q: 27  
b: 27  
y: 27  
l: 26  
k: 25  
-----
```

```
-----  
2-gram (top 20):
```

```
dc: 39  
ax: 35  
rz: 30  
zd: 29  
nx: 29  
mn: 24  
nc: 23  
xu: 21  
du: 20  
dx: 19  
xa: 17  
ez: 17  
rd: 17  
ds: 17  
ns: 16  
pc: 16  
xd: 15  
xn: 15  
ud: 15  
nv: 15  
-----
```

```
-----  
3-gram (top 20):
```

```
rzd: 21  
nxu: 14  
cmn: 10  
nez: 9  
dcn: 9  
zdc: 9  
ddx: 8  
axd: 8  
mnx: 8  
axw: 8  
epm: 8  
dxa: 7  
xaw: 7  
awm: 7  
wmn: 7  
mne: 7  
eza: 7  
zax: 7  
wdc: 7  
udv: 7
```

I wrote a script called freq.py that will read the ciphertext.txt file and filter out non=letter characters. It would count the occurrences of n-grams. 1- gram, the most frequent letters in 1-

gram are d, n, and x. 2- gram, the most frequent bigrams are dx, an, and rz. 3- gram, the most frequent trigrams are rzd, nxu, and cmn. When looking at the numbers I am able to try and convert the most found grams into their original states. For example, the trigram rzd was found 21 times and when converted back it would spell the word "the." Even though it looks extremely confusing at first, it is a little insane that you can decipher letter by letter possibly just by trying to find commonalities or repetitions of certain letters or words that would usually be so common in normal speech.

Task 5) Understanding MD5 Collision Properties

1. Used md5sum to compare the hashes of two different files.
2. Observed that the files produced the same MD5 hash.
3. Used the cat command to combine the files.
4. Generated a new hash for the combined file.

```
matt@matt-VirtualBox:~/Downloads$ md5sum file1.txt file2.txt
eafcf30a135dfbe24bb9d6d5401ec99e  file1.txt
eafcf30a135dfbe24bb9d6d5401ec99e  file2.txt
matt@matt-VirtualBox:~/Downloads$ cat file1.txt file2.txt > file3.txt
matt@matt-VirtualBox:~/Downloads$ md5sum file3.txt
16e4d38eae21a9265b526408a75a4973  file3.txt
```

We first use md5sum to observe that these two different inputs have the same MD5 hash. I am then able to understand that they are a collision pair and that I can use cat to combine them into one file. It is interesting though to see that two inputs that had the same md5sum combined give me a file with a new MD5 hash.

Task 6) Creating a Program for Collision Injection

1. Created a C file named task3.c.
2. Initialized a 400-byte array filled with the character 'A'.
3. Compiled the program using gcc to create an executable.
4. Verified the program printed hexadecimal values.

```
matt@matt-VirtualBox:~/Downloads$ vim task3.c
```

```
#include <stdio.h>
```

```
unsigned char xyz[400] = {
    [0 ... 399] = 'A'
};
```

```
int main() {
    int i;
    for (i=0; i<400; i++) {
        printf("%x", xyz[i]);
    }
    printf("\n");
    return 0;
}
```

```
matt@matt-VirtualBox:~/Downloads$ gcc task3.c -o task3
```

```
matt@matt-VirtualBox:~/Downloads$ ./task3
```

[illegible]

I wrote a C program that initializes a 400-byte array with value 'A.' This will then print the hex value values of that array, which is 41 in this case.

Task 7) Splitting the Executable File

1. Used head to extract the prefix of the executable.
2. Used tail to extract the suffix.
3. Ensured the prefix length was a multiple of 64 bytes.

```
matt@matt-VirtualBox:~/Downloads$ head -c 12352 task3 > prefix
matt@matt-VirtualBox:~/Downloads$ tail -c +12481 task3 > suffix
matt@matt-VirtualBox:~/Downloads$ wc -c task3 prefix suffix
17184 task3
12352 prefix
 4704 suffix
34240 total
```

Task 8: Generating the MD5 Collision

1. Ran the md5collgen tool on the prefix.
2. Generated two files named t1.bin and t2.bin.
3. Verified both files had the same MD5 hash.

```
matt@matt-VirtualBox:~/Downloads$ ./md5collgen -p prefix -o t1.bin t2.bin
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 't1.bin' and 't2.bin'
Using prefixfile: 'prefix'
Using initial value: 421159c10dfd1e0594fad958ed32f904

Generating first block: ....
.....
Generating second block: S10.....
Running time: 4.98741 s

matt@matt-VirtualBox:~/Downloads$ md5sum t1.bin t2.bin
005ccdd0d0e51869a05a467d5e711113  t1.bin
005ccdd0d0e51869a05a467d5e711113  t2.bin
```

I used the md5collgen tool on the prefix file. I had to add the “./” before the md5collgen since it wasn’t allowing me to use md5collgen alone. This tool generated two different outputs, t1.bin and t2.bin. They have the same MD5 hash.

Task 9: Building the Final Executables

1. Extracted the last 128 bytes from each collision file.
2. Appended the prefix, collision block, and suffix together.
3. Created two final executables: a1.out and a2.out.
4. Verified hashes using md5sum and sha256sum.

```
matt@matt-VirtualBox:~/Downloads$ tail -c 128 t1.bin > P
matt@matt-VirtualBox:~/Downloads$ tail -c 128 t2.bin > Q
matt@matt-VirtualBox:~/Downloads$ wc -c P Q
128 P
128 Q
256 total
```



```
matt@matt-VirtualBox:~/Downloads$ cat prefix P suffix > a1.out
matt@matt-VirtualBox:~/Downloads$ cat prefix Q suffix > a2.out
matt@matt-VirtualBox:~/Downloads$ md5sum a1.out a2.out
c847b86da1d15834ead09140dad4baa0  a1.out
c847b86da1d15834ead09140dad4baa0  a2.out
matt@matt-VirtualBox:~/Downloads$ sha256sum a1.out a2.out
527a538d50b7b2a47a6eda8fae89ea4a108de683fb8a270e0a235921a860367d  a1.out
fc35bd9b18e6498560187a2c352117ecebdc28af9886e0ee06e879038d997611  a2.out
```

I created two executables by combining the pieces. I then double checked their hashes. The MD5 hashes were the same but the sha256 hashes were different. This showed me how if an attacker created a program with a malicious intent and it had the same MD5 hash as another normal and probably beneficial program, it would be so easy to bypass security systems that use MD5 for any type of verification. Sha256 showed the difference between the two and why MD5 is too weak and unsafe for today's confidentiality and authentication, when it comes to modern society's technological knowledge of cybersecurity.